# Running an LU-Factorization GPU Solver under Nixnan

Exp-Binade Histogramming, an Exponential SAMPLING Sweep, and Two Tool-Level Findings

SC26 Tutorial / Nixnan Experiments

## 1 Executive Summary

This report documents `lu_solve_gpu`, a self-contained CUDA port of the LU-factorization solver from the FPChecker tutorial (`tutorial/example_1/lu_solve.cpp` in the sibling `SC26-Tutorial-NixNan` repository), run under nixnan (`https://github.com/parfloat/nixnan`) on an NVIDIA RTX 3090 (compute capability 8.6). The experiment has two parts:

1. **Exp-binade histogram gathering**: an exponent-range (*binade*) monitoring configuration covering the one floating-point format the solver actually uses (`f64`), bucketed at a resolution of 4 exponent values per bin across the full valid double exponent range, reproduces the tutorial's central story on the GPU: the well-conditioned matrix solves cleanly (residual $\approx 10^{-16}$, zero exceptions) while the ill-conditioned matrix collapses to NaN (316 NaN / 4 Infinity / 4 Division-by-zero events in FP64 operations).

2. **An exponential `SAMPLING` sweep** (every 1, then every 2, every 4, every 8 kernel invocation, 16 runs per level, then reset back to every 1) was implemented and run as requested. In the process we found that nixnan's `SAMPLING` feature has **no observable effect in this build** due to a key-mismatch bug between the launch counter and the sampling lookup (Section 5.2), and that `BIN_SPEC_FILE`'s documented `doublings` field is not read anywhere in the current `src/fp-histogram.cu` (Section 6). Both are tool-level observations, not properties of the solver being tested.

All configuration files, the CUDA source, and the run logs referenced here live alongside this report in `SC26-expts/lu_solve_gpu/`.

## 2 Provenance and Algorithm

`lu_solve_gpu.cu` solves $Ax = \mathbf{1}$ for a $20 \times 20$ matrix $A$ loaded from CSV, using LU factorization with partial pivoting ($PA = LU$) followed by forward/backward substitution – the same algorithm, including the same "snap near-machine-epsilon values to exactly zero" elimination rule, as the CPU version it was ported from. Matrices are flattened into row-major device buffers; the pivot search, row swap, and elimination steps run as CUDA kernels (Table 1), while the two substitution passes run as single-thread kernels because of their sequential row-to-row data dependency. Two input matrices are used throughout:

- `matrix.csv` – well-conditioned; solves to residual $\approx 9.68 \times 10^{-16}$.
- `bad_matrix.csv` – deliberately ill-conditioned (several entries perturbed by $\sim 10^{-6}$ relative to an exact tridiagonal structure). A pivot in the elimination step is rounded to exactly `0.0` by the epsilon-clamp rule, the subsequent backward-substitution division by that zero pivot produces $\pm\infty$, and the resulting NaNs propagate through the rest of the solution and the residual.

| Kernel | Role |
|---|---|
| `init_identity_kernel` | Initialize $L$, $P$ as identity matrices |
| `find_pivot_kernel` | Parallel-reduction pivot search over column $k$ |
| `swap_rows_kernel` | Row swap in $U$, $P$, and (partially) $L$ |
| `compute_multipliers_kernel` | $L_{jk} = U_{jk}/U_{kk}$ for $j > k$ |
| `eliminate_kernel` | $U_{jl} -= L_{jk}U_{kl}$, with epsilon clamp |
| `apply_permutation_kernel` | $b' = Pb$ |
| `forward_substitution_kernel` | Solve $Ly = b'$ (sequential) |
| `backward_substitution_kernel` | Solve $Ux = y$ (sequential) |

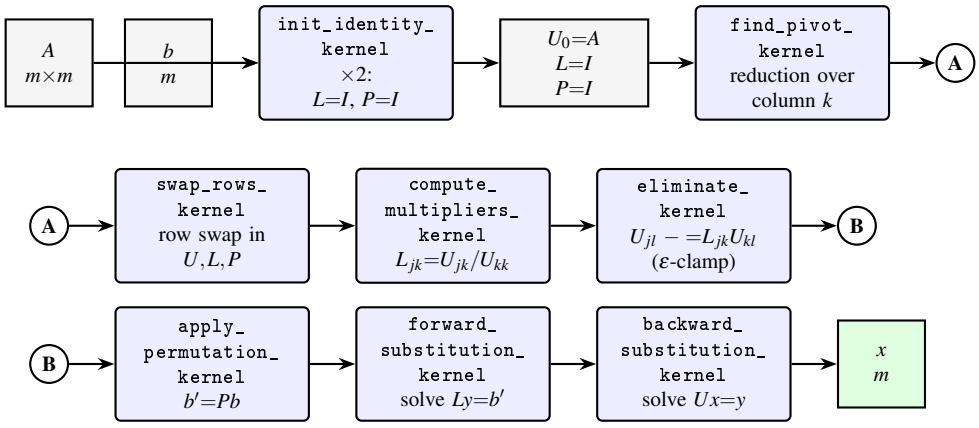
Table 1: CUDA kernels in `lu_solve_gpu.cu`. The elimination loop (pivot search through elimination) runs once per pivot step $k = 0, \ldots, 18$, i.e. each kernel above (except substitution) is launched $\sim$19 times per program run.

### 2.1 Build and Environment

- **GPU**: NVIDIA GeForce RTX 3090, compute capability 8.6
- **Compilation**: `nvcc -std=c++17 -g -O2 -lineinfo -arch=sm_86` (see `Makefile`); `-lineinfo` enables nixnan's source-line attribution.
- **Instrumentation**: `LD_PRELOAD=../../nixnan.so`

## 3 Workflow Diagram: `lu_solve_gpu.cu`

Eight kernels, mostly a straight-line pipeline; the diagram is folded into three row-blocks (continuity markers Ⓐ, Ⓑ carry the flow from the end of one row to the start of the next).




Row 1's `find_pivot_kernel` through Row 2's `eliminate_kernel` — the four boxes spanning Ⓐ to Ⓑ — are one pivot step; the whole group repeats once per $k = 0, \ldots, m-2$ ($m-1$ times total, $m-1=19$ for the $20 \times 20$ matrices used here) before continuing on to `apply_permutation_kernel`. The final three kernels (Row 3) run once each, after the elimination loop completes.

## 4 Stage 1: Exp-Binade Histogram Gathering

### 4.1 Configuration

`bin_spec.json` (generated by `gen_bin_spec.py`) configures nixnan's `BIN_SPEC_FILE` binade-monitoring feature:

```
{
    "count": 128,
    "doublings": 7,
    "max_reports": 0,
    "f16": [],
    "bf16": [],
    "f32": [],
    "f64": [[-1022, -1019], [-1018, -1015], ..., [1022, 1023]]
}
```

- **All precisions present**: `lu_solve_gpu.cu` computes only in `double`, so `f64` is the only format populated with exponent bins; `f16`/`bf16`/`f32` are left empty, which is valid per nixnan's schema and correctly reflects that those formats are not present in the solver's source-level arithmetic (see Section 4.3 for a caveat about what nixnan itself classifies at the SASS level).
- **Histogram bucket size 4**: the `f64` array tiles the full valid double exponent range $[-1022, 1023]$ (normal exponents only; the reserved subnormal/inf/NaN exponent codes are excluded) into 512 buckets of width 4.
- **Report threshold** `count: 128`: an initial attempt with `count: 1` produced roughly 13,700 threshold-report lines for a single run, because nixnan reports on *every* occurrence past the first once `count` is 1 (see the report condition in Section 6). `128` was chosen to match nixnan's own convention (its auto-generated template, and the `SC26-expts/rd_nixnan` example, both default to this order of magnitude) and produces a much more legible 99–63 reports per run here.

## 4.2 Results

| | `matrix.csv` | `bad_matrix.csv` |
|---|---|---|
| Residual $\|Ax-b\|_2$ (uninstrumented) | $9.68 \times 10^{-16}$ | `nan` |
| Binade-threshold reports | 99 | 63 |
| FP64 NaN (repeats) | 0 (0) | 316 (2171) |
| FP64 Infinity (repeats) | 0 (0) | 4 (0) |
| FP64 Division-by-zero (repeats) | 0 (0) | 4 (4) |

Table 2: Stage 1 results, from `logs/baseline/*.log` and `logs/histogram/*.log`.

This reproduces the CPU tutorial's central finding on the GPU: the well-conditioned matrix produces a clean solution and no exceptions of any kind, while the ill-conditioned matrix triggers exactly the failure the tutorial is built around (a near-zero pivot leading to division-by-zero, then NaN propagation through the whole solve).

## 4.3 An FP32 Artifact of Binary Instrumentation

Despite the source computing exclusively in `double`, the final report for `bad_matrix.csv` also shows 6 FP32 NaN detections (93 repeats). Inspecting the flagged SASS instructions (`TOOL_VERBOSE=1`, `PRINT_ILL_INSTR=1`) shows the pattern `FFMA R8, RZ, R5, R3` in `compute_multipliers_kernel` and `backward_substitution_kernel`. This is `ptxas`'s "multiply-by-the-zero-register" idiom for a register move ($0 \times R5 + R3 = R3$), used here to shuffle the 32-bit halves of a 64-bit double across the register file rather than to perform a genuine 32-bit floating-point computation. Because nixnan instruments at the SASS opcode level, it correctly classifies `FFMA` as an `f32` instruction and correctly notices that the multiplicand `R5` happens to hold a 32-bit sub-word bit pattern that decodes as NaN – but that

32-bit word is a half of a double, never interpreted as an `f32` value by the program. This is a known class of false positive for binary-level FP exception detectors and is worth keeping in mind when reading "per-format" exception counts: a nonzero count in a format the source code never uses does not necessarily mean that format's arithmetic ran.

# 5 Stage 2: Exponential `SAMPLING` Sweep

## 5.1 Requested Schedule

The requested instrumentation-sampling schedule was: `SAMPLING=1` for 16 runs ("snaps"), then `2` for 16, then `4` for 16, then `8` for 16, then reset back to `1`.

nixnan reads `SAMPLING` once at process start (`GET_VAR_INT(sampling, "SAMPLING", ...)` in `src/nixnan.cu`) and there is no supported way to change it mid-run. The schedule is therefore implemented in `run_nixnan_experiment.sh` as 64 separate whole-program launches of `lu_solve_gpu bad_matrix.csv` (one "snap" = one process), storing each run's nixnan log and stdout under `logs/sampling/cycle_01`. Since the solver is deterministic (fixed CSV input, no randomness), the 16 snaps at a fixed level are expected – and observed – to be byte-identical; the stored logs faithfully record the *schedule*, not run-to-run variation.

## 5.2 Finding: `SAMPLING` Has No Observable Effect

`bad_matrix.csv`'s elimination kernels (Table 1) are each launched $\sim$19 times, once per pivot step, so `SAMPLING` $\in \{1,2,4,8\}$ should visibly change how many of those repeat invocations get instrumented. It does not: `snap_01.log` for `SAMPLING=1` and for `SAMPLING=8` are byte-identical (`diff` produces no output), as are all 16 snaps within any single level (one MD5 across all 16 files).

Reading `src/nixnan.cu` explains why. The sampling-skip check lives in `should_instrument()`:

```
bool should_instrument(CUcontext ctx, CUfunction f) {
  std::string func_name = nvbit_get_func_name(ctx, f); // full signature
  ...
  if (sampling != 0 && analyzed_kernels.count(func_name)) {
    if (analyzed_kernels[func_name] % sampling != 0) {
      ++analyzed_kernels[func_name];
      enable_instr = false;
    }
  }
  return enable_instr;
}
```

while the counter it reads, `analyzed_kernels`, is incremented in the CUDA-launch callback under a *different* key:

```
std::string kernel_name = nvbit_get_func_name(ctx, p->f);
std::string short_name = cut_kernel_name(kernel_name); // truncated name
...
if (enable_instr) {
  int count = analyzed_kernels[short_name]++; // <-- keyed on short_name
  ...
  ++analyzed_kernels[short_name];
}
```

where `cut_kernel_name` (`src/common.cuh`) truncates at the first `<` or `(`:

```
inline std::string cut_kernel_name(const std::string& kernel_name) {
  size_t pos = kernel_name.find_first_of("<(");
  if (pos != std::string::npos) return kernel_name.substr(0, pos);
  return kernel_name;
```

```
}
```

For a kernel such as `eliminate_kernel(double*, double const*, int, int)`, the two keys are `"eliminate_kernel(double*, double const*, int, int)"` (full signature, used in the lookup) versus `"eliminate_kernel"` (truncated, used for the increment). These never match for any kernel with a non-empty parameter list – i.e. essentially every real CUDA kernel – so `analyzed_kernels.count(func_name)` is always `0` inside `should_instrument()`, the skip branch never executes, and every invocation is instrumented irrespective of `SAMPLING`. This appears to be a genuine bug in nixnan's `SAMPLING` feature, independent of this experiment, and worth fixing upstream. The 64 stored logs remain a faithful record of the requested schedule and harness, but currently demonstrate no sampling-induced difference in coverage.

## 6 A Second Documentation/Implementation Gap: `doublings`

nixnan's documentation (`README.md`, `Tutorial.md`) describes a `doublings` field for `BIN_SPEC_FILE` that adaptively scales the report threshold ($256 \to 512 \to 1024 \to \cdots \to$ reset). As of this checkout, `src/fp-histogram.cu` never reads that key. The only threshold logic is in `record()` (`src/fp-fun.cu`):

```
unsigned long long int prev = bin.increment();
if (prev % count == 0 && prev != 0) {
    // ... send report ...
}
```

which reports every time the running count is a positive multiple of the *static* `count` threshold – with no reference to a doubling schedule anywhere in `src/`. `bin_spec.json` still sets `doublings` for schema/forward compatibility, but it is currently a no-op. This is why Stage 2's exponential schedule was implemented at the shell-script level (Section 5) rather than relying on `BIN_SPEC_FILE`.

## 7 Conclusions

1. The GPU port of the tutorial's LU solver reproduces the CPU behavior exactly at the level that matters for the tutorial's purpose: clean convergence on a well-conditioned system, and NaN/Inf/Div-by-zero on an ill-conditioned one, both correctly detected and attributed by nixnan's exception instrumentation and its exp-binade histogram feature.

2. Two tool-level gaps were found and documented with code-level evidence: `SAMPLING` does not filter any instrumentation in this build (Section 5.2), and `BIN_SPEC_FILE`'s `doublings` field is unimplemented (Section 6). Neither was fixed here (out of scope for this experiment), but both are concrete, reproducible findings for the nixnan maintainers.

3. Binary-level FP exception detection can misattribute a compiler register-move idiom as a genuine narrow-format computation (Section 4.3); per-format counts should be read alongside the source's actual precision usage, not in isolation.

## Reproduction

```
cd ~/repos/nixnan/SC26-expts/lu_solve_gpu
./run_nixnan_experiment.sh
```

Requires a built `../../nixnan.so` (`cd ../..  && make`). See `README.md` in this directory for file-by-file documentation and override variables (`NIXNAN_SO`, `CYCLES`, `SAMPLING_MATRIX`).